

Projet Ingénierie des réseaux

Sujet 4 : Prédiction de trafic

Younes Osman
Ilias Berkhoussi
Vincent Dambrine
Youssef Bali
Zakaria Benihoud

2SN-R Mai 2020

Ce projet vise à comparer les performances d'algorithmes de prédiction de trafic (fondées sur des algorithmes de type AR, NN, SDM,...). L'objectif est de comparer ces méthodes en termes de complexité et de précision de la prédiction du débit agrégé sur la voie aller ou sur la voie retour.

L'objectif final de la prédiction est d'adapter en temps réel les algorithmes d'allocation de ressources. Par exemple, sur la voie aller, allouer des faisceaux et adapter les emplacements des empreintes au sol en fonction du débit agrégé des utilisateurs sur la voie aller. Sur la voie retour la prédiction peut servir à optimiser la boucle d'allocation des ressources.

L'impact de l'horizon de la prédiction est à étudier : peut-on faire une prédiction fiable à l'échelle de la seconde, de la minute de la dizaine de minutes ?

Les données sont fournies selon un scénario de communication précis et au cas échéant peuvent être générées par un outil de simulation (ns-2).

Le choix du code des algorithmes de prédiction est laissé libres (mais un code Matlab peut également être fourni).

Sommaire :

Présentation des différents types d'algorithmes	3
AR - Auto Regressive	3
NN - Neural Networks	5
SVM - Support Vector Machine	6
Comparaison des algorithmes	8
Méthodes de ML vs Méthodes Statistiques	8
Démonstration & Résultats	11
Conclusion	12

Présentation des différents types d'algorithmes

On parle ici de l'importance de ces algorithmes dans le contexte de la prédiction de trafic. Le but de la prédiction de trafic dans le réseau est de prévoir les observations futures sur la quantité de trafic.

Il y a une grande dépendance entre les futures observations $X_n + k$ et les valeurs prises du passé $X_0, X_1, \dots, X_{n-1}, X_n$. Et pour obtenir une bonne prédiction, il est important d'utiliser le modèle approprié. Par la suite, on pourra en faire une implémentation facile avec un minimum de paramètres à estimer.

Pour des processus à longue mémoire, une bonne prédiction à long et à court terme peut être obtenue avec l'existence d'un grand nombre d'enregistrements pris du passé. Pour avoir une meilleure prédiction, il faut se baser sur les critères suivants :

- Bien choisir le modèle de prédiction pour fournir une grande exactitude
- Afin d'achever une prévision à temps réel, il faut avoir une certaine simplicité dans le choix des paramètres à utiliser et à implémenter
- Un bon modèle de prédiction doit s'adapter à tout changement de trafic.

Différents outils ont été proposés pour effectuer les prédictions sur les processus à longue mémoire (LRD). Les principaux modèles retenus sont les processus ARIMA.

Par leur simplicité et flexibilité, ces modèles sont devenus très populaires dans leur application dans l'analyse des séries de temps (séries chronologiques). On va alors commencer par présenter cette algorithmique.

AR - Auto Regressive

Les types d'algorithmes Auto régressifs les plus souvent utilisés sont les algorithmes ARMA.

L'algorithme ARMA (Auto-Regressive Moving Average) est un des algorithmes les plus couramment utilisés pour modéliser des séries chronologiques univariées (c'est à dire, unidimensionnel). On peut modéliser le problème en une dimension en fonction du temps).

$ARMA(p, q)$ combine deux composantes : $AR(p)$ et $MA(q)$.

Dans le modèle $AR(p)$, la valeur Y_n d'une série temporelle peut être estimée en utilisant une combinaison linéaire des p dernières observations. On utilise aussi un terme d'erreur e_n et un terme constant c .

Ce qui nous donne la première équation:

$$Y_n = c + \sum_i (\phi_i * Y_{n-i}) + en$$

Où ϕ_i , $\forall i \in \{1, \dots, p\}$ dénote les paramètres du modèle, et p représente l'ordre du modèle.

De même, le modèle $MA(q)$ utilise les erreurs précédentes comme des variables explicatives :
Ce qui nous donne la deuxième équation:

$$Y_n = \mu + \sum_i (\theta_i * Y_{n-i}) + en$$

Où μ dénote la moyenne des observations θ_i , $\forall i \in \{1, \dots, q\}$, qui représente les paramètres du modèles, et q qui représente l'ordre du modèle.

On a alors $ARMA(p, q)$ qui peut être construit en combinant les deux modèles et on a la troisième équation:

$$Y_n = c + \sum_i (\phi_i * Y_{n-i}) + \sum_i (\theta_i * Y_{n-i}) + en$$

Il faut noter que $ARMA(p, q)$ est défini pour des données stationnaires.

Cependant, beaucoup de phénomènes physiques dans le monde réel n'ont pas de structure stationnaire, par exemple, lorsque la série est influencée par des saisons ou par une tendance générale.

Le modèle $ARIMA(p, d, q)$ vient alors compléter le modèle $ARMA(p, q)$ en ajoutant un paramètre d'intégration d'ordre d .

Essentiellement, ARIMA fonctionne en appliquant des transformations de différenciation d à la série temporelle (jusqu'à ce qu'elle devienne stationnaire), avant d'appliquer $ARMA(p, q)$.

Contrairement aux autres algorithmes qu'on va présenter par la suite, l'algorithme $ARMA(p, q)$ est le seul qui ne fait pas partie de la famille des algorithmes de machine learning.

NN - Neural Networks

Les algorithmes NN (Neural Networks : de réseau de neurones) sont, comme leur nom l'indique, basés sur un processus d'évolution génétique.

Les algorithmes NN font partie de la famille des algorithmes issus du machine learning.

En effet, avec ces algorithmes, la machine apprend elle-même à traiter des données. Ici, on l'utilise pour prédire du trafic.

Lors de recherche dans la littérature, nous avons trouvés que les types d'algorithmes NN utilisés pour la prédiction de trafic sont nombreux.

On en cite deux ici: ANN et RNN.

ANN : Une fois le processus d'apprentissage fait (long et laborieux), on a une utilisation quasi-instantanée (contrairement aux algos ARMA ou ARIMA).

RNN : Ils lient les neurones de la fin à ceux du début, créant un cycle, qui sert en quelque sorte de mémoire artificielle. C'est très utile pour étudier les données temporelles, qui dépendent souvent de l'état "n-1" du système.

Ils utilisent le Jordan Neural Network qui est simple, et le loop qui sert de mémoire s'appelle le "context layer", il contient la sortie n-1 du NN.

SVM - Support Vector Machine

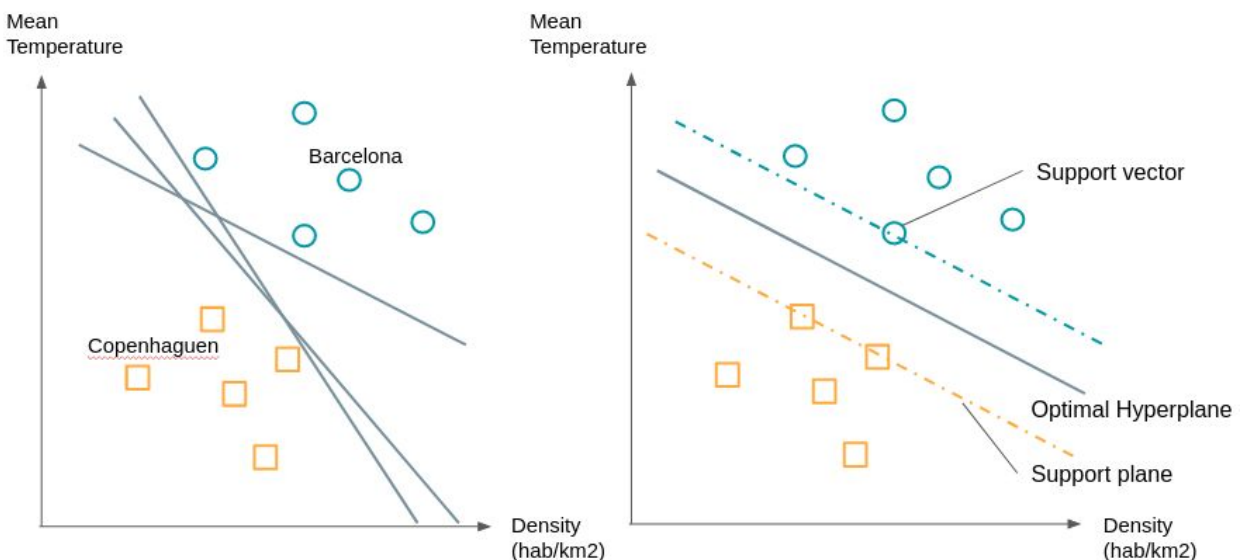
Aussi connu sous le nom de Machines à Vecteurs de Support cet algorithme sert principalement à des problèmes de classification même si il a été étendu à des problèmes de régression.

Pour mieux comprendre cet algorithme on va prendre un exemple. Pour la simplicité de notre exemple considérons seulement 2 variables pour décrire chaque ville : la température et la densité de population. On peut donc représenter les villes en 2 dimensions.

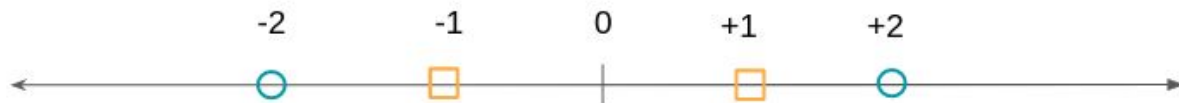
Nous représentons par des ronds des villes que vous avez beaucoup aimé visiter et par des carrés celles que vous avez moins apprécié. Lorsque vous considérez de nouvelles villes vous souhaitez savoir de quelle groupe ces villes se rapproche-t-elle le plus.

Comme nous le voyons sur le graphique de droite il existe de nombreux plans (des droites lorsque nous n'avons que 2 dimensions) qui sépare les deux groupes.

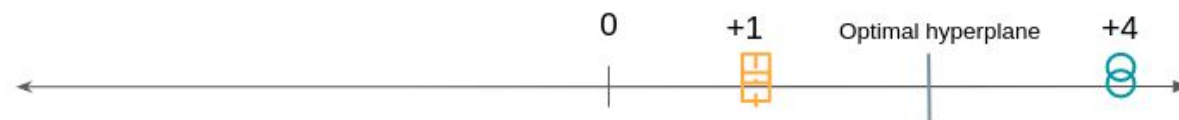
On va choisir la droite qui est à la distance maximale entre les deux groupes. Pour le construire nous voyons déjà que n'avons pas besoin de tous les points, il suffit de prendre les points qui sont à la frontière de leur groupe on appelle ces points ou vecteurs, les vecteurs supports. Les plans passant par ces vecteurs supports sont appelés plans supports. Le plan de séparation sera celui qui sera équidistant des deux plans supports.



Before : no possible separation



After : (transformation $f(x) = x^2$)



L'algorithme SVM va donc consister à chercher à la fois l'hyperplan optimal ainsi que de minimiser les erreurs de classification.

Comparaison des algorithmes

Méthodes de ML vs Méthodes Statistiques

Dans cette étude nous allons comparer les méthodes suivantes :

Méthodes de Machine Learning:

RBR, RF, GP, MARS et GLM.

Méthodes Statistiques:

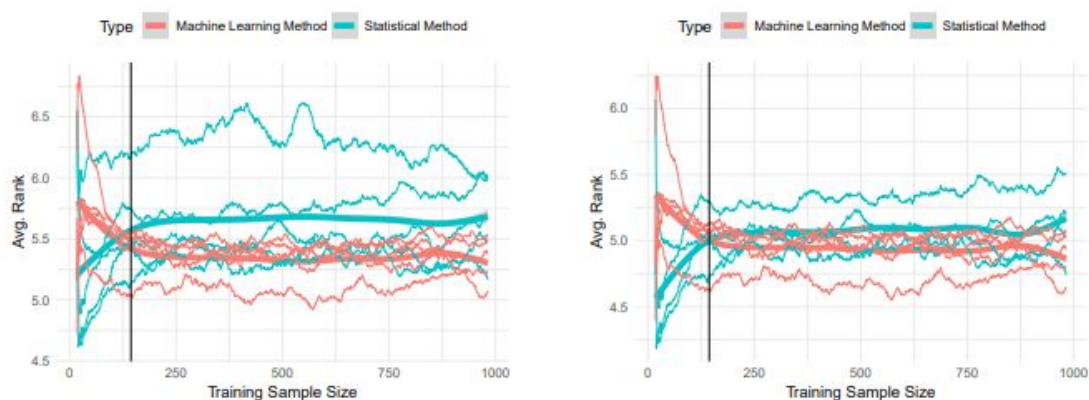
ARIMA, Naive2, Theta, ETS, Tbats.

Les méthodes d'apprentissage automatique sont de plus en plus utilisées pour résoudre les problèmes de prévision de séries chronologiques univariées.

En effet, Une étude présente une analyse des différentes méthodes d'apprentissage automatique pour cette tâche en utilisant des séries temporelles. Leurs résultats suggèrent que les méthodes du perceptron multicouche (machine learning) et des processus gaussiens présentent les meilleures performances prédictives. Cependant, les auteurs de cette étude ne comparent pas ces méthodes avec des approches de pointe, telles que l'ARIMA ou le lissage exponentiel.

Dans une autre étude de cas, on a comparé différents modèles de prévision, y compris des méthodes statistiques et des méthodes d'apprentissage automatique. Dans leur analyse, les dernières approches présentent un meilleur classement moyen (meilleure performance prédictive) par rapport aux premières. Cette dernière étude a été prolongée en incluant plusieurs méthodes statistiques dans leur configuration expérimentale. Leurs résultats suggèrent que la plupart des méthodes statistiques surpassent systématiquement les méthodes d'apprentissage automatique pour la prévision de séries chronologiques univariées.

Cet effet est perceptible pour les prévisions en une étape et en plusieurs étapes. Les méthodes d'apprentissage automatique analysées par les auteurs incluent différents types de réseaux de neurones (par exemple, une mémoire à court terme à long terme, un perceptron multicouche), la méthode des voisins les plus proches, un arbre de décision, une régression vectorielle de soutien et des processus gaussiens. D'un autre côté, les méthodes statistiques incluent l'ARIMA, le lissage exponentiel naïf et le thêta, entre autres.



Les résultats représentés sur cette figure montrent une tendance claire:

Lorsque seules quelques observations sont disponibles, les méthodes statistiques présentent de meilleures performances. Cependant, à mesure que la taille de l'échantillon augmente, les méthodes d'apprentissage automatique les surpassent.

À savoir, que les modèles statistiques surpassent ceux de l'apprentissage automatique, lorsque nous ne considérons que les ensembles de formation jusqu'à **144 observations**.

Nous incluons également une analyse de la complexité de calcul de chaque méthode. Nous évaluons cela en fonction du temps de calcul passé par un modèle, que nous définissons comme le temps nécessaire à un modèle pour terminer la procédure de la prédiction séquentielle. Nous définissons la complexité de calcul (CC) d'un modèle m comme suit:

$$CC = \text{Temps de calcul } m / \text{Temps de calcul Naive2}$$

Essentiellement, nous normalisons le temps de calcul de chaque méthode par le temps de calcul de la méthode Naive2. Les résultats sont présentés sur la figure 7 sous forme de graphique à barres. Les barres du graphique sont mises à l'échelle logarithmique. La valeur d'origine avant de prendre le logarithme est affichée dans chaque barre.

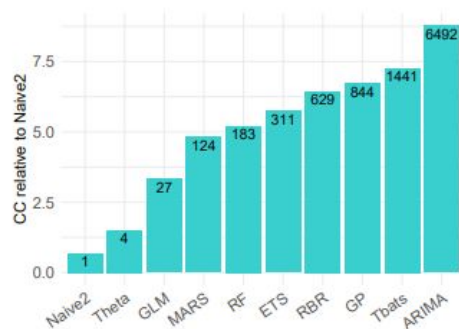


Figure 7: Computational complexity of each method relative to the Naive2 benchmark model (log scaled)

Démonstration & Résultats

On a comparé trois algorithmes :

- AR
- ARMA
- NN

Les implémentations ont été faites sous matlab.

Nous avons d'abord fait une prédiction à court terme : 20 secondes.

On voit alors que le trafic prédit (courbe colorée) correspond au vrai trafic (courbe noire).

Les prédictions d'AR et d'ARMA "recopient intelligemment" les valeurs du passé, mais c'est ce qui était attendu.

Le réseau de neurone, lui, essaie vraiment de deviner les pics.

En somme dans ce cas, les résultats sont précis, surtout dans le cas de AR et ARMA.

A long terme, les algorithmes AR et ARMA donnent surtout une moyenne, qui, pour le coup, est proche du trafic réel. Malheureusement, ces débits ne retranscrivent pas les pics de trafics. C'est justement ici la force des réseaux de neurones : l'algorithme NN essaie de deviner les pics de trafic, et même si des fois il se trompe, il arrive relativement bien à les prédire.

Il faut noter que lorsque le régime n'est plus stationnaire, c'est à dire quand la moyenne du trafic change, l'algorithme NN est perdu.

Ainsi, il faudra soit coupler l'algorithme NN avec un algorithme statistique (comme ARMA par exemple), soit ré-entraîner périodiquement le réseau de neurone, tel que l'échantillon d'entraînement soit similaire aux valeurs à prédire.

Nous pouvons aussi faire un point sur les temps d'exécution des différents algorithmes :

- L'algorithme AR, le plus simple est aussi le plus rapide : 0.57 secondes d'exécution.
- L'algorithme ARMA prends un peu plus de temps : 1.73 secondes. Il reste néanmoins rapide vu les ordres de grandeurs de prédiction.
- L'algorithme NN prends le plus de temps : 13.32 secondes, apprentissage compris. Il faut noter que la majorité de ce temps vient de l'apprentissage. Sans apprentissage, l'exécution de cet algorithme est très rapide : de l'ordre de la seconde.

Ainsi, même via les temps d'exécution on voit que les algorithmes statistiques (AR et ARMA) sont plus adaptés aux prédictions à court terme, tandis que l'algorithme NN est plus adapté aux prédictions à long terme.

Conclusion

L'impact de l'horizon de la prédiction est à étudier : peut-on faire une prédiction fiable à l'échelle de la seconde, de la minute de la dizaine de minutes ?

A courte échelle (de l'ordre de quelques secondes), les méthodes statistiques l'emportent. En effet, à cette échelle de temps, on peut s'attendre à ce que le trafic n'augmente ou ne diminue pas subitement.

Ainsi, se baser sur les valeurs du passé et espérer qu'elles soient similaires dans le futur est une bonne approche.

Ensuite, pour les estimations à l'échelle de la minute, il vaut mieux utiliser les réseaux de neurones qui sont plus apte à prédire les pics d'affluence (ce que les méthodes statistiques ne peuvent pas faire).

Il faut néanmoins nuancer cette réponse : si les valeurs de trafic deviennent trop éloignées de celles d'apprentissages, le réseau de neurone va prédire des valeurs moins précises que les méthodes statistiques.